

Transparency Primitives for Proof-Carrying Anonymity Claims: Beacons, VRFs, and Append-Only Logs (Series Synthesis)

Unified archive note v0.03 (2026-02-27; archive v100)

Abstract

Several papers in this archive rely on the same publication/anti-equivocation substrate: (i) a public randomness source (beacon or VRF), (ii) a precommitment digest for a release/evaluation manifest, and (iii) an append-only transparency log with inclusion and consistency proofs. To keep those papers brief and non-redundant, this note states the minimal interface and security properties assumed. It is deliberately lightweight: we are not proposing new primitives, only an audit-friendly *contract* that existing systems (Certificate Transparency-style logs, Sigstore/Rekor-like services, and beacon/VRF sources) can satisfy.

Scope. This note specifies the *assumed interface* for public randomness and append-only publication used by the evaluation and release-workflow papers.

Imports. We treat cryptographic constructions as black boxes and cite representative standards (VRFs and CT-style logs). The artifact-policy note covers what is shipped inline vs. on-request.[1]

Exports. We export (i) a minimal beacon/VRF binding contract, (ii) an append-only log contract (entries, checkpoints, inclusion/consistency proofs), and (iii) a small set of failure modes that must be priced in threat-window declarations.

1 Public randomness: beacons and VRFs

Many evaluation workflows need randomness that the evaluator cannot grind after seeing outcomes. We assume a *public randomness interface* that yields values that are (a) unpredictable before a declared time and (b) publicly verifiable after publication.

Definition 1 (Beacon interface). *A beacon is a public function that maps a round identifier r to an output $R_r \in \{0,1\}^\ell$ together with verification data. The interface provides: (i) $\text{VerifyBeacon}(r, R_r, \pi_r) \in \{0,1\}$ and (ii) a public schedule indicating when R_r becomes available. Security assumptions:*

- **(Unpredictability)** *Before the disclosure time of round r , no efficient party can predict R_r with non-negligible advantage.*
- **(Public verifiability)** *Anyone can verify that (r, R_r) is valid.*

Examples include drand-style beacons and NIST randomness beacons.[5, 6]

Definition 2 (VRF interface). *A VRF provides a keypair (pk, sk) and algorithms $\text{Eval}(\text{sk}, m) \rightarrow (y, \pi)$ and $\text{Verify}(\text{pk}, m, y, \pi) \in \{0,1\}$, where y is pseudorandom conditioned on pk and m . Security assumptions:*

- (**Uniqueness**) For a fixed (pk, m) there is at most one valid output y (prevents equivocation).
- (**Pseudorandomness**) Without sk , y is indistinguishable from random given (pk, m) .

See, e.g., the VRF standard.[3]

2 Binding: manifest digests to evaluation seeds

The core anti-grinding pattern is: *commit first, then derive randomness from a public source.*

Definition 3 (Canonical digest). *A manifest (release plan, evaluation plan, parameter list) is mapped to a byte string by a canonicalization rule and hashed:*

$$D = H(\text{Canonicalize}(\text{manifest})).$$

The only requirement is that independent verifiers compute the same digest D from the published manifest bytes.[4]

Definition 4 (Seed binding). *Given a manifest digest D and a public randomness value U (beacon output R_r or VRF output y), define the evaluation seed*

$$\text{Seed} = H(D \parallel U \parallel \text{domain_sep}),$$

*where **domain_sep** prevents cross-protocol reuse. A valid certificate must disclose (D, U, Seed) and the verification data for U .*

Remark 1 (Threat-window implications). *If an evaluator can choose U after observing partial outcomes (e.g., select the beacon round post-hoc), grinding reappears. Threat-window declarations should therefore treat “choice of U ” as a control-plane action that must be precommitted and logged.*

3 Append-only transparency logs

A transparency log is a public, append-only data structure that supports efficient proofs of inclusion and append-only consistency.

Definition 5 (Log interface). *A log accepts entries e and returns: (i) an inclusion proof π_{inc} that e is included at tree size n under root h_n , and (ii) periodic checkpoints (tree size n , root h_n , timestamp) signed by the log operator. The interface supports **VerifyInclusion** $(e, n, h_n, \pi_{\text{inc}})$ and **VerifyConsistency** $(n_1, h_{n_1}, n_2, h_{n_2}, \pi_{\text{con}})$ for append-only growth. Certificate Transparency-style Merkle-tree logs are the canonical example.[2]*

Definition 6 (Anti-equivocation goal). *A log is anti-equivocating for the purposes of this archive if: (i) checkpoints are widely witnessed/monitored (or cosigned) so split views are detectably inconsistent, and (ii) certificates carry enough log evidence (checkpoint + inclusion proof) that verifiers can test consistency against the public view. This is a deployment/interface goal rather than a new cryptographic definition.[7, 8]*

What gets logged? For this archive, a typical entry is a tuple

$$e = (\text{kind}, D, U, \text{metadata}),$$

where `kind` distinguishes release manifests from evaluation attempts, and `metadata` includes the declared spending schedule id, attempt index, *exposure normal form id* (the transcript projection being budgeted), and pointer(s) to on-request artifacts.

Remark 2 (Surface binding is part of anti-grinding). *A log entry that binds code, parameters, and randomness but omits the claim surface still leaves room for semantic drift: the publisher can later present the same run as a claim about a different transcript projection. For certificate-carrying papers in this archive, the minimal safe rule is therefore: if a budget is claimed on a projection X , then the logged statement must hash-bind a stable identifier for that projection (an exposure normal-form id / schema hash). This is what makes later “same claim versus new claim” checks mechanically meaningful.*

4 Failure modes to price

The interface above is small, but several failure modes must be priced explicitly:

- **Split view / inconsistent checkpoints.** Mitigate with witness policies, monitors, or cosigned checkpoints.
- **Post-hoc randomness choice.** Require precommitment of beacon round / VRF message and log it before outcomes.
- **Canonicalization ambiguity.** Require a single canonical JSON/text scheme and publish the exact bytes that were hashed.

These are not “paper cuts”: they are exactly the channels by which stop-when-pass workflows re-enter.

5 Takeaways

- **Commit first, randomize second:** derive evaluation seeds from a manifest digest and a public randomness source.
- **Logs are the anti-abort primitive:** publish every initiated attempt (including aborts) to make grinding auditable.
- **Specify the interface:** papers stay brief if they can import a shared log/beacon contract rather than re-defining it.

References

- [1] Anonymous. Artifact policy (source-only archive; artifacts available on request). Series note (Synthesis 6), 2026.
- [2] IETF. RFC 9162: Certificate Transparency Version 2.0. 2021.
- [3] IETF. RFC 9381: Verifiable Random Functions (VRFs). 2023.

- [4] IETF. RFC 8785: JSON Canonicalization Scheme (JCS). 2020.
- [5] drand project. Distributed randomness beacon. Public documentation, 2020–2026.
- [6] NIST. NIST Randomness Beacon. Public documentation, 2013–2026.
- [7] C2SP. Transparency log witness policy / witness ecosystem notes. Public drafts, 2023–2026.
- [8] C. Newman et al. Sigstore: Software signing for everybody. Public report / paper, 2022.